



Python subprocess module

Last Updated : 4 Apr, 2026

The subprocess module is used to run external programs or system commands from Python. It allows to execute commands, capture their output and interact with other processes. One can also send input, handle errors and check execution status.

Example: In this example, a system command is executed to check the Python version and print its output.

```
import subprocess
res = subprocess.run(["python", "--version"], capture_output=True, text=True)
print(res.stdout)
```

Loading Playground...

Output

Python 3.13.0

Explanation:

- subprocess.run() executes the command
- capture_output=True stores output instead of printing directly
- text=True converts output to string
- result.stdout contains the output

subprocess.run()

subprocess.run() executes a command and waits until it finishes. It returns an object that contains output, errors and return status of the command.

Example: Here, a command is executed and both output and return code are printed.

```
import subprocess
res = subprocess.run(["python", "--version"], capture_output=True, text=True)
print("Output:", res.stdout)
print("Return Code:", res.returncode)
```

Loading Playground...

Output

Output: Python 3.13.0

Return Code: 0

- `subprocess.run()` runs the command and waits
- `res.returncode` gives execution status (0 means success)
- `res.stdout` stores command output

subprocess.Popen

Popen starts a new process and allows interaction with it while it is running. It gives control over input, output and error streams.

Example: In this example, a process is started and its output is captured.

```
import subprocess

process = subprocess.Popen(
    ["python", "--version"],
    stdout=subprocess.PIPE,
    text=True
)

output, _ = process.communicate()
print(output)
```

Output

Python 3.13.0

Explanation:

- `subprocess.Popen()` starts the process
- `stdout=subprocess.PIPE` captures output
- `process.communicate()` reads output and waits for completion

subprocess.check_output

`check_output()` executes a command and returns only its output. If the command fails, it raises an exception.

Example: Here, a command is executed and its output is directly returned.

```
import subprocess

try:
    output = subprocess.check_output(["python", "--version"], text=True)
    print(output)
except subprocess.CalledProcessError:
    print("Command failed")
```

Output

- `subprocess.check_output()` runs the command
- Returns output directly and raises error if execution fails

subprocess.call

`subprocess.call()` runs a command and returns its exit status. It does not store output in variables.

Example: In this example, a command is executed and its success status is checked.

```
import subprocess
code = subprocess.call(["python", "--version"])

if code == 0:
    print("Success")
else:
    print("Failed")
```

Output

```
Python 3.13.0
Success
```

Explanation:

- `subprocess.call()` executes the command
- Returns 0 if successful
- Output is printed directly to terminal

subprocess Pipes

Pipes are used to send input to a process and receive output from it. This helps in communication between processes.

Example: Here, input is sent to a Python process and the result is captured.

```
import subprocess
process = subprocess.Popen(
    ["python"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True
)

output, _ = process.communicate("print(5 + 5)")
print(output)
```

Output

Explanation:

- `stdin=subprocess.PIPE` allows sending input
- `stdout=subprocess.PIPE` captures output
- `process.communicate()` sends input and reads result

Comment

S Sourav...

18

Article Tags:

[Python](#)

[Python-Library](#)

[python-modules](#)



Corporate & Communications Address:

A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company

[About Us](#)

[Legal](#)

[Privacy](#)

[Policy](#)

[Careers](#)

[Contact Us](#)

[Corporate](#)

[Solution](#)

[Campus](#)

[Training](#)

Explore

[POTD](#)

[Practice](#)

[Problems](#)

[Blogs](#)

[Upskill](#)

[Courses](#)

[Connect](#)

Tutorials

[Programming](#)

[Languages](#)

[DSA](#)

[Web](#)

[Technology](#)

[AI, ML &](#)

[Data Science](#)

[DevOps](#)

[CS Core](#)

[Subjects](#)

[GATE](#)

[School](#)

[Subjects](#)

[Software and](#)

[Tools](#)

Courses

[ML and Data](#)

[Science](#)

[DSA and](#)

[Placements](#)

[Web](#)

[Development](#)

[Data Science](#)

[Programming](#)

[Languages](#)

[DevOps &](#)

[Cloud](#)

[GATE](#)

[MongoDB](#)

[Certifications](#)

Preparation Corner

[Interview](#)

[Corner](#)

[Aptitude](#)

[Puzzles](#)

[GfG 160](#)

[System Design](#)



@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved